LAPORAN RISET

MODUL 4
“PEMROGRAMAN OTA BERBASIS BLYNK”

oleh :

Evan Adwitiya Dwi Listanto / 23081010093

ROBOTIKA A-801

PROGRAM STUDI INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS PEMBANGUNAN NASIONAL “VETERAN”

JAWA TIMUR

2026

1. Tujuan

Percobaan ini bertujuan untuk :

1. Mengonfigurasi template perangkat pada Blynk.Console (Template ID, Template Name, datastream V0 sampai V6, dan dashboard) sebagai identitas ESP32 pada Kit iMCLab yang menggunakan kerangka Blynk.Edgent.
2. Menjalankan provisioning kredensial WiFi dan Auth Token secara otomatis melalui aplikasi Blynk, sehingga kredensial tersebut tidak perlu dituliskan langsung di dalam kode program.
3. Memperbarui firmware ESP32 dari versi 0.1.0 ke versi 0.1.1 secara nirkabel (Over-The-Air) melalui layanan Blynk.Air tanpa kabel USB.
4. Mengonfirmasi keberhasilan proses OTA melalui pengecekan nomor versi firmware serta fungsi-fungsi baru—Lampu, Mode Kedip, dan Kecepatan Kedip—yang tersedia pada dashboard Blynk.
5. Menganalisis alur kerja kode Blynk.Edgent, terutama mesin status (state machine), mode provisioning, dan proses pengunduhan firmware yang terdapat pada berkas OTA.h.

2. Dasar Teori

1. **Over-The-Air (OTA).** OTA adalah metode pembaruan firmware perangkat tertanam melalui jaringan nirkabel, tanpa menghubungkan perangkat ke komputer menggunakan kabel USB. Firmware baru berupa berkas biner (.bin) diunduh oleh mikrokontroler, ditulis ke memori flash, kemudian perangkat dimulai ulang untuk menjalankan versi baru. Pada ESP32, proses ini didukung oleh tabel partisi yang memiliki dua slot aplikasi (app0 dan app1) serta partisi otadata yang mencatat slot yang aktif. Firmware baru ditulis ke slot yang sedang tidak digunakan sehingga versi lama tetap utuh selama proses berlangsung. Manfaat utama OTA adalah efisiensi pemeliharaan perangkat yang tersebar atau sulit dijangkau secara fisik.

2. **Blynk IoT dan Template.** Blynk adalah platform IoT berbasis cloud yang menghubungkan perangkat seperti ESP32 dengan dashboard pada aplikasi mobile maupun web (Blynk.Console). Pada Modul 3, perangkat diidentifikasi oleh Template ID, Template Name, dan Auth Token yang ditulis langsung di dalam kode. Template menyimpan definisi datastream (Virtual Pin) dan tampilan dashboard yang diwarisi oleh setiap perangkat yang dibuat dari template tersebut. Template ID selalu berawalan "TMPL" dan menjadi kunci yang menghubungkan firmware dengan template di cloud.

3. **Blynk.Edgent.** Blynk.Edgent adalah kerangka sketch siap pakai dari Blynk yang memuat empat fungsi utama, yaitu Blynk.Inject (provisioning kredensial WiFi secara dinamis), Blynk.Air (pembaruan firmware OTA), indikator status perangkat melalui LED, dan penghapusan kredensial melalui tombol fisik. Kode Edgent terbagi menjadi sketch utama (.ino) dan sejumlah berkas header (.h) yang masing-masing menangani satu fungsi. Karena Auth Token diberikan secara otomatis saat provisioning, sketch Edgent tidak boleh mendefinisikan BLYNK_AUTH_TOKEN.

4. **Provisioning (Blynk.Inject).** Provisioning adalah proses memberikan kredensial kepada perangkat yang belum dikonfigurasi, yaitu SSID, kata sandi WiFi, dan Auth Token. Pada Blynk.Inject, ESP32 yang belum terkonfigurasi akan membuat jaringan WiFi sendiri (mode Access Point). Aplikasi Blynk pada smartphone terhubung ke jaringan tersebut, lalu mengirimkan kredensial melalui permintaan HTTP. Kredensial disimpan pada memori flash non-volatil (NVS) sehingga tetap tersimpan setelah perangkat dimatikan, dan perangkat berpindah ke mode Station untuk terhubung ke router dan cloud Blynk.

5. **Blynk.Air dan Pembaruan Firmware.** Blynk.Air adalah layanan OTA Blynk yang diakses melalui menu Developer Zone > Blynk.Air (OTA) pada Blynk.Console. Pengguna mengunggah berkas .bin, memilih perangkat

tujuan, lalu memulai pengiriman (shipping). Blynk Cloud mengenali firmware melalui tag yang tertanam di dalam biner, antara lain Template ID dan nomor versi, sehingga nomor versi harus dinaikkan setiap kali firmware baru dibuat. Perangkat menerima alamat unduhan dari cloud, mengunduh biner melalui HTTP, memverifikasi checksum MD5, menuliskannya ke partisi OTA, dan melakukan restart.

6. Alat, Bahan, dan Konfigurasi Pin

Perangkat Keras yang digunakan dalam Percobaan ini :

- ESP 32
- Modul motor IMCLab
- Kabel USB Data
- Kabel Power 5V 2 A
- Laptop
- Smartphone

Perangkat Lunak yang digunakan dalam Percobaan ini :

- Arduino IDE Versi 2.3.10
- Aplikasi Blynk IoT di HP
- Library Blynk Versi 1.3.5
- Blynk Console di browser
- Versi core ESP32 di Boards Manager : 2.0.17
- Jaringan Wifi

Tabel 1 Pin pada ESP32

Fungsi di kode	Variabel di kode	GPIO di kode	Label di papan
LED indikator	ledPin	2	D2
Pin kontrol arah motor 1	motor1Pin1	27	D27
Pin kontrol arah motor 2	motor1Pin2	26	D26
Enable (PWM 30 kHz, 8 bit)	enable1Pin	12	D12
Sensor RPM (interrupt FALLING)	pin_rpm	13	D13

Tabel 2. Datastream pada template

Nama	Pin	Tipe data	Min	Maks	Satuan	Arah dan fungsi
RPM	V0	Double	0	13000	rpm	Perangkat ke dashboard: kecepatan putar (Gauge)
PWM	V1	Integer	0	255	BIT	Perangkat ke dashboard: nilai PWM aktual (Gauge)
PWM MOTOR	V2	Integer	0	255	BIT	Dashboard ke perangkat: slider PWM
Power Switch	V3	Integer	0	1	-	Dashboard ke perangkat: saklar motor ON/OFF
Lampu	V4	Integer	0	1	-	Dashboard ke perangkat: saklar LED menyala terus
Mode Kedip	V5	Integer	0	1	-	Dashboard ke perangkat: saklar LED berkedip

Nama	Pin	Tipe data	Min	Maks	Satuan	Arah dan fungsi
Kecepatan Kedip	V6	Integer	1	10	-	Dashboard ke perangkat: slider (1 pelan, 10 cepat)

7. Prosedur Percobaan

1. Buka website Blyn.Console melalui browser dan masuk menggunakan akun Blynk yang sudah terdaftar.
2. Pilih Menu Developer zone > My Template > New Template
3. Isi nama template dengan “IMCLab OTA”, pilih Hardware ESP32, tentukan tipe koneksi WiFi, lalu klik Done.
4. Pada halaman template, buka tab Info (General Settings) dan catat nilai Template ID dan Template Name. Kedua nilai ini harus sama persis dengan yang tertulis pada kode program
5. Buka tab Datastreams, klik “+ New Datastream”, kemudian pilih Virtual Pin.
6. Buat Datastream pertama (V0) dengan nama "RPM", ", atur Data Type menjadi Integer, nilai Min 0 dan Max 3000.
7. Buat Datastream kedua (V1) dengan nama "Kecepatan PWM", atur Data Type menjadi Integer, nilai Min 0 dan Max 255.
8. Buat Datastream ketiga (V2) dengan nama "PWM motor", atur Data Type menjadi Integer, nilai Min 0 dan Max 255.
9. Buat Datastream pertama (V3) dengan nama "Power Switch", atur Data Type menjadi Integer, nilai Min 0 dan Max 1.
10. Buat Datastream kedua (V4) dengan nama "Lampu", atur Data Type menjadi Integer, nilai Min 0 dan Max 1.
11. Buat Datastream ketiga (V5) dengan nama "Mode Kedip", atur Data Type menjadi Integer, nilai Min 0 dan Max 1.
12. Buat Datastream pertama (V6) dengan nama "Kecepatan Kedip", atur Data Type menjadi Integer, nilai Min 0 dan Max 10.
13. Klik Save pada masing-masing datastream untuk memastikan Virtual Pin (V0 hingga V6) tersimpan dengan benar

14. Pada halaman template di Blynk.Console, buka tab Web Dashboard (atau Mobile Dashboard jika menggunakan aplikasi HP). Klik ikon Edit (berbentuk pensil) di pojok kanan atas untuk mulai mengatur tampilan.
15. Klik tombol + (Add Widget) atau kotak kosong pada area dashboard untuk memunculkan menu pilihan widget, lalu pilih Gauge.
16. Klik widget Gauge yang baru ditambahkan, lalu pada menu pengaturan di sebelah kanan, atur Datastream ke V0 (RPM). Ubah judul widget menjadi "RPM" dan atur mode tampilan menjadi Gauge
17. Tambahkan widget baru dengan klik tombol + lagi, lalu pilih Radial Gauge. Atur Datastream ke V1 (PWM), beri judul " PWM", dan atur nilai Min = 0 serta Max = 255.
18. Tambahkan widget ketiga dengan klik +, lalu pilih Slder lalu pilih Slider. Atur Datastream ke V2 (PWM Motor), beri judul " PWM", dan atur nilai Min = 0 serta Max = 255.
19. Tambahkan widget keempat dengan klik +, lalu pilih Switch. Atur Datastream ke V3 (Power Switch), beri judul " Power Switch ", dan atur mode tampilan menjadi Switch (ON/OFF).
20. Tambahkan widget keempat dengan klik +, lalu pilih Switch. Atur Datastream ke V4 (Lampu), beri judul " Daya Lampu ", dan atur mode tampilan menjadi Switch (ON/OFF).
21. Tambahkan widget keempat dengan klik +, lalu pilih Switch. Atur Datastream ke V5 (Mode Kedip), beri judul " Mode Kedip ", dan atur mode tampilan menjadi Switch (ON/OFF).
22. Tambahkan widget ketiga dengan klik +, lalu pilih Slder lalu pilih Slider. Atur Datastream ke V6 (Kecepatan Kedip), beri judul "Kecepatan Kedip", dan atur nilai Min = 0 serta Max = 10.
23. Setelah ketujuh widget selesai ditambahkan, atur tata letak (posisi dan ukuran) masing-masing widget dengan cara drag and drop agar tampilan dashboard rapi dan mudah digunakan.

24. Klik tombol Save (ikon disket) di pojok kanan atas untuk menyimpan perubahan. Pastikan tampilan dashboard sudah sesuai sebelum diuji coba pada perangkat keras.
25. Hubungan Modul ESP 32 Ke Laptop menggunakan kabel USB data.
26. Pastikan Kabel USB yang digunakan mendukung Transfer data, bukan hanya untuk pengisian daya.
27. Periksa Apakah ESP32 sudah terdeteksi pada komputer.
28. Buka aplikasi Arduino IDE.
29. Pada menu bar, pilih File > Examples > Blynk > Blynk.Edgent > ESP32_Edgent
30. Simpan sketch contoh tersebut dengan nama baru. Pilih menu File > Save As..., lalu beri nama folder dan file menjadi Blynk_ota.
31. Pada kode Blynk_ota.ino yang sudah terbuka, cari dan sesuaikan baris berikut agar sesuai dengan template yang ada di Blynk.Console

```
#define BLYNK_TEMPLATE_ID           "TMPxxxxxx"
#define BLYNK_TEMPLATE_NAME         "Device"
```

32. Pilih board ESP32 melalui menu Tools > Board > esp32 > DOIT ESP32 DEVKIT V1 .
33. Pilih port melalu menu Tools > Port >(Pilih port yang sesuai).
34. Klik Tombol Upload
35. Tunggu hingga proses upload selesai.
36. Setelah proses upload selesai, buka Serial Monitor (ikon kaca pembesar di pojok kanan atas) dan atur baud rate menjadi 115200.
37. Amati banner yang muncul di Serial Monitor. Layar akan menampilkan versi firmware (0.1.0) dan status awal perangkat (biasanya menampilkan instruksi untuk melakukan konfigurasi WiFi).
38. Setelah proses upload selesai, buka Serial Monitor (ikon kaca pembesar di pojok kanan atas) dan atur baud rate menjadi 115200.
39. Amati banner yang muncul di Serial Monitor. Layar akan menampilkan versi firmware (0.1.0) dan status awal perangkat (biasanya menampilkan instruksi untuk melakukan konfigurasi WiFi).
40. Buka Blynk.Console > Devices dan pastikan perangkat berstatus Online dengan versi firmware 0.1.0

41. Buka kembali Arduino IDE, lalu pilih File > Examples > Blynk > Blynk.Edgent > ESP32_Edgent
42. Simpan sketch contoh tersebut dengan nama baru. Pilih menu File > Save As..., lalu beri nama folder dan file menjadi Program_kedip_Lampu.
43. Hapus seluruh kode bawaan example tersebut, lalu salin dan tempel kode beriut

```
/*
DATASTREAM pada template IMCLab OTA (semua harus dibuat di template)
    V0  RPM             Double   0 sampai 13000   Gauge    (perangkat
ke dashboard)
    V1  PWM             Integer  0 sampai 255     Gauge    (perangkat
ke dashboard)
    V2  PWM MOTOR       Integer  0 sampai 255     Slider   (dashboard
ke perangkat)
    V3  Power Switch    Integer  0 sampai 1       Switch   (dashboard
ke perangkat)
    V4  Lampu           Integer  0 sampai 1       Switch
    V5  Mode Kedip      Integer  0 sampai 1       Switch   (1 = LED
berkedip)
    V6  Kecepatan Kedip  Integer  1 sampai 10     Slider   (1 pelan,
10 cepat)
*/

// Harus sama dengan template tempat perangkat ini diaktivasi (lihat
Blynk.Console)
#define BLYNK_TEMPLATE_ID           "TMPxxxxxx"
#define BLYNK_TEMPLATE_NAME         "Device"

// Naikkan nomor ini SETIAP kali membuat versi baru untuk OTA
#define BLYNK_FIRMWARE_VERSION "0.1.1"

#define BLYNK_PRINT Serial
#define APP_DEBUG

#include "BlynkEdgent.h"

// 1 = aktifkan kontrol lewat Serial Monitor (perintah 1, 0, +, -,
PWM:xxx, R)
// 0 = nonaktif. Edgent punya konsol serial sendiri; jika perintah
terbaca terpotong
// atau tidak dikenali, atur kembali ke 0.
#define SERIAL_KONTROL 0

// ================================================================
// PIN
// ================================================================
const int LED_PIN = 2;                  // LED lampu/kedip: LED biru
bawaan ESP32 (D2)

// LED merah di papan (dekat resistor 220 ohm). Nomor pinnya belum
diketahui.
// Cari dengan Cari_Pin_LED_Merah.ino, lalu isi di sini. -1 = tidak
dipakai.
const int  LED_MERAH_PIN       = -1;
const bool LED_MERAH_AKTIF_LOW = false;    // true jika LED merah
menyala saat pin LOW
```

```
// true  = saat Lampu dan Mode Kedip mati, LED mengikuti status motor
(seperti kode asli)
// false = LED hanya dikontrol Lampu dan Mode Kedip
const bool LED_MENUNJUKKAN_MOTOR = false;

const int motor1Pin1 = 27;            // kontrol arah motor 1 (label
D27)
const int motor1Pin2 = 26;            // kontrol arah motor 2 (label
D26)
const int enable1Pin = 12;            // Enable, keluaran PWM (label
D12)
const int pin_rpm    = 13;             // sensor RPM (label D13)

// ============================================================
// PWM MOTOR
// ============================================================
const int freq       = 30000;
const int pwmChannel = 0;
const int resolution = 8;

const int DUTY_STEP = 10;
const int DUTY_MIN  = 0;
const int DUTY_MAX  = 255;

int  dutyCycle   = 0;                  // motor mati saat mulai
bool motorEnable = false;

// ============================================================
// RPM
// ============================================================
const int HOLES = 2;
const unsigned long DEBOUNCE_US = 500;

volatile unsigned long rev = 0;
volatile unsigned long lastPulseTime = 0;
float rpm = 0.0;
unsigned long old_time = 0;

// ============================================================
// LED
// ============================================================
bool lampOn    = false;               // switch Lampu (V4)
bool blinkMode = false;               // switch Mode Kedip (V5)
int  speedVal  = 5;                   // kecepatan kedip 1..10 (V6)
bool ledState  = false;
unsigned long lastToggle = 0;

String inputString = "";

// ============================================================
// PWM (mendukung core ESP32 versi 2.x dan 3.x)
// ============================================================
void pwmSetup() {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
  ledcAttach(enable1Pin, freq, resolution);
#else
  ledcSetup(pwmChannel, freq, resolution);
  ledcAttachPin(enable1Pin, pwmChannel);
#endif
}

void pwmWrite(int duty) {
#if ESP_ARDUINO_VERSION_MAJOR >= 3
```

```
  ledcWrite(enable1Pin, duty);
#else
  ledcWrite(pwmChannel, duty);
#endif
}

// ================================================================
// ISR sensor RPM
// ================================================================
void IRAM_ATTR isr() {
  unsigned long now = micros();
  if (now - lastPulseTime > DEBOUNCE_US) {
    rev++;
    lastPulseTime = now;
  }
}

// ================================================================
// MOTOR
// ================================================================
void applyMotor() {
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  pwmWrite(dutyCycle);

  // Sinkronisasi ke dashboard
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

void tampilkanStatus(const char* aksi) {
  Serial.printf("%s | PWM=%d | RPM=%.1f\n", aksi, dutyCycle, rpm);
}

// Hitung RPM: pulsa dihitung 1 detik, rpm = (pulsa / jumlah lubang)
x 60
void hitungRPM() {
  detachInterrupt(digitalPinToInterrupt(pin_rpm));
  unsigned long n = rev;
  rev = 0;
  attachInterrupt(digitalPinToInterrupt(pin_rpm), isr, FALLING);
  rpm = ((float)n / HOLES) * 60.0;
}

void kirimKeBlynk() {
  if (!Blynk.connected()) return;
  Blynk.virtualWrite(V0, rpm);
  Blynk.virtualWrite(V1, dutyCycle);
  Blynk.virtualWrite(V2, dutyCycle);
  Blynk.virtualWrite(V3, dutyCycle > 0);
}

// V2: slider PWM dari Blynk
BLYNK_WRITE(V2) {
  dutyCycle   = constrain(param.asInt(), DUTY_MIN, DUTY_MAX);
  motorEnable = (dutyCycle > 0);
  applyMotor();
  Serial.printf("PWM dari Blynk = %d\n", dutyCycle);
}

// V3: saklar ON/OFF dari Blynk
BLYNK_WRITE(V3) {
```

```
  motorEnable = param.asInt();
  if (motorEnable) {
    if (dutyCycle == 0) dutyCycle = 200;
  } else {
    dutyCycle = 0;
  }
  applyMotor();
  Serial.printf("Motor %s\n", motorEnable ? "ON" : "OFF");
}

// ============================================================
// LED
// ============================================================
// Kecepatan kedip 1..10 menjadi jeda: 1 -> 1000 ms (pelan), 10 ->
100 ms (cepat)
unsigned long blinkInterval() {
  return map(speedVal, 1, 10, 1000, 100);
}

void setLeds(bool on) {
  digitalWrite(LED_PIN, on ? HIGH : LOW);
  if (LED_MERAH_PIN >= 0) {
    bool level = LED_MERAH_AKTIF_LOW ? !on : on;
    digitalWrite(LED_MERAH_PIN, level ? HIGH : LOW);
  }
}

BLYNK_WRITE(V4) {
  lampOn = param.asInt();
  Serial.printf("Lampu: %s\n", lampOn ? "ON" : "OFF");
}

BLYNK_WRITE(V5) {
  blinkMode  = param.asInt();
  lastToggle = millis();
  Serial.printf("Mode kedip: %s\n", blinkMode ? "ON" : "OFF");
}

BLYNK_WRITE(V6) {
  speedVal = constrain(param.asInt(), 1, 10);
  Serial.printf("Kecepatan kedip: %d (jeda %lu ms)\n", speedVal,
blinkInterval());
}

// Prioritas: Mode Kedip, lalu Lampu, lalu (opsional) status motor.
// Tidak memakai delay() supaya Blynk dan OTA tetap berjalan.
void updateLed() {
  bool nyala;
  if (blinkMode) {
    if (millis() - lastToggle >= blinkInterval()) {
      lastToggle = millis();
      ledState = !ledState;
    }
    nyala = ledState;
  } else if (lampOn) {
    nyala = true;
  } else if (LED_MENUNJUKKAN_MOTOR) {
    nyala = (dutyCycle > 0);
  } else {
    nyala = false;
  }
  setLeds(nyala);
}
```

```
// ============================================================
// KONTROL SERIAL (opsional, lihat SERIAL_KONTROL di atas)
// ============================================================
#if SERIAL_KONTROL
void prosesPerintahSerial() {
  while (Serial.available() > 0) {
    char c = Serial.read();

    if (c == '\n' || c == '\r') {
      inputString.trim();

      if (inputString.startsWith("PWM:")) {
        int   val   =   constrain(inputString.substring(4).toInt(),
DUTY_MIN, DUTY_MAX);
        dutyCycle   = val;
        motorEnable = (dutyCycle > 0);
        applyMotor();
        tampilkanStatus("Set PWM");

      } else if (inputString.length() > 0) {
        char cmd = inputString.charAt(0);

        if (cmd == '1') {
          if (dutyCycle == 0) dutyCycle = 200;
          motorEnable = true;
          applyMotor();
          tampilkanStatus("Motor ON");

        } else if (cmd == '0') {
          dutyCycle   = 0;
          motorEnable = false;
          applyMotor();
          Serial.println("Motor OFF");

        } else if (cmd == '+') {
          dutyCycle += DUTY_STEP;
          if (dutyCycle > DUTY_MAX) dutyCycle = DUTY_MAX;
          motorEnable = true;
          applyMotor();
          tampilkanStatus("PWM NAIK");

        } else if (cmd == '-') {
          dutyCycle -= DUTY_STEP;
          if (dutyCycle < DUTY_MIN) dutyCycle = DUTY_MIN;
          motorEnable = (dutyCycle > 0);
          applyMotor();
          tampilkanStatus("PWM TURUN");

        } else if (cmd == 'R' || cmd == 'r') {
          Serial.printf("RPM = %.1f | PWM = %d\n", rpm, dutyCycle);
        }
      }
      inputString = "";
    } else {
      inputString += c;
    }
  }
}
#endif

// ============================================================
// KONEKSI
```

```
// ============================================================
// Saat tersambung (atau tersambung ulang), ambil nilai terakhir dari
dashboard
BLYNK_CONNECTED() {
  Blynk.syncVirtual(V2, V3, V4, V5, V6);
}

void setup() {
  Serial.begin(115200);
  delay(100);

  pinMode(LED_PIN, OUTPUT);
  if (LED_MERAH_PIN >= 0) pinMode(LED_MERAH_PIN, OUTPUT);

  pinMode(motor1Pin1, OUTPUT);
  pinMode(motor1Pin2, OUTPUT);
  pinMode(enable1Pin, OUTPUT);
  pinMode(pin_rpm, INPUT_PULLUP);

  pwmSetup();
  pwmWrite(0);                                    // motor mati saat
mulai
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  attachInterrupt(digitalPinToInterrupt(pin_rpm), isr, FALLING);

  old_time = millis();

  Serial.println("iMCLab    +    Blynk    Edgent    -    versi    "
BLYNK_FIRMWARE_VERSION);
#if SERIAL_KONTROL
  Serial.println("Perintah: 1=ON, 0=OFF, +=naik, -=turun, PWM:xxx,
R=lihat RPM");
#endif
  BlynkEdgent.begin();
}

void loop() {
  BlynkEdgent.run();
  updateLed();

  if (millis() - old_time >= 1000) {
    old_time = millis();
    hitungRPM();
    kirimKeBlynk();
  }

#if SERIAL_KONTROL
  prosesPerintahSerial();
#endif
}
```

44. Pada kode `Program_kedip_Lampu.ino` yang sudah terbuka, cari dan sesuaikan baris berikut agar sesuai dengan template yang ada di Blynk.Console

```
#define BLYNK_TEMPLATE_ID           "TMPxxxxxx"
#define BLYNK_TEMPLATE_NAME         "Device"
```

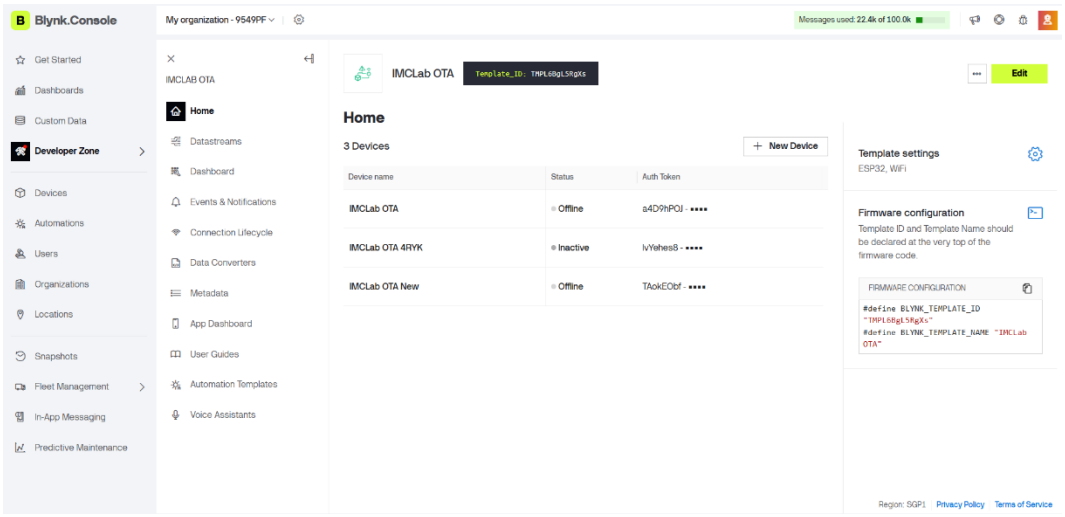
45. Klik Verify untuk mengompilasi, lalu pilih menu Sketch > Export Compiled Binary

46. Pastikan perangkat ESP32 berstatus Online dan saklar Power pada dashboard berada dalam posisi OFF.
47. Buka Blynk.Console, lalu pilih menu Developer Zone > Blynk.Air (OTA).
48. Pilih perangkat tujuan pada Devices list.
49. Unggah berkas Program_kedip_Lampu.ino.bin pada area unggah firmware, lalu periksa Firmware info yang dibaca oleh Blynk Cloud (tipe, versi, dan template).
50. Klik Start Shipping
51. Pantau status pengiriman pada daftar Shipping di Blynk.Console
52. Tunggu hingga perangkat melakukan restart dan kembali berstatus Online, kemudian periksa versi firmware yang tampil
53. Pada dashboard Blynk, ketuk Switch Lampu ke posisi ON dan amati LED biru pada ESP32, lalu kembalikan ke OFF.
54. Aktifkan Switch Mode Kedip, lalu geser Slider Kecepatan Kedip ke nilai 1, 5, dan 10. Amati dan catat perubahan kecepatan kedip LED.
55. Aktifkan Mode Kedip bersamaan dengan Lampu dan amati LED untuk mengetahui fungsi yang diprioritaskan.
56. Nyalakan Switch Power, atur Slider PWM ke nilai 200, lalu catat nilai RPM dan PWM pada gauge dan bandingkan dengan hasil Modul 3
57. Opsional: pada Serial Monitor (pilih line ending Newline), ketik firmware info atau devinfo untuk menampilkan versi firmware melalui konsol bawaan Edgent.

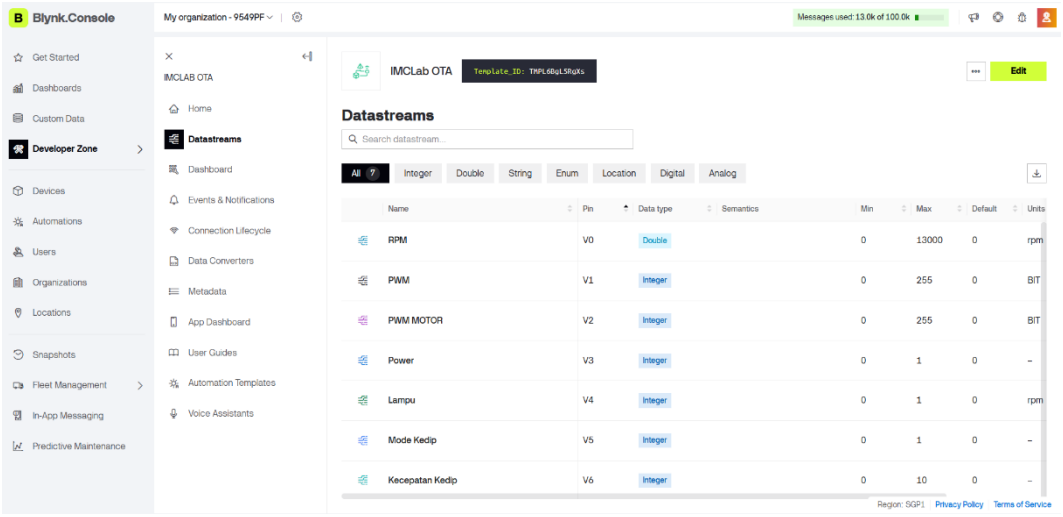
5. Hasil Percobaan

1. Konfigurasi Template dan Dashboard Blynk

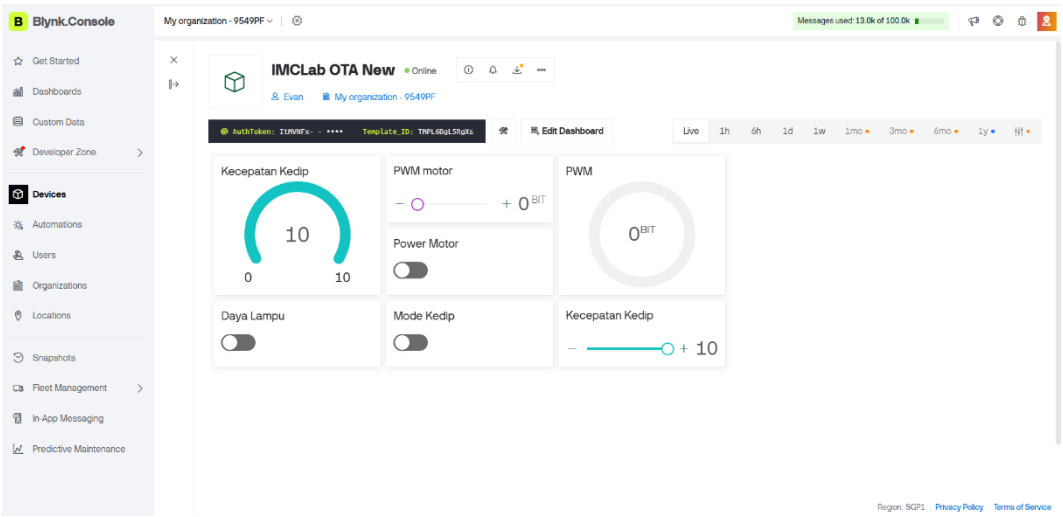
   Template “IMCLab OTA” berhasil dibuat pada Blynk.Console dengan Template ID, seperti terlihat pada Gambar 1. Tujuh datastream (V0 sampai V6) berhasil didaftarkan sesuai Tabel 2 dan ditampilkan pada Gambar 2. Dashboard web yang memuat dua gauge, dua slider, dan tiga switch ditunjukkan pada Gambar 3.



Gambar 1. Halaman Info template "IMCLab OTA" pada Blynk.Console



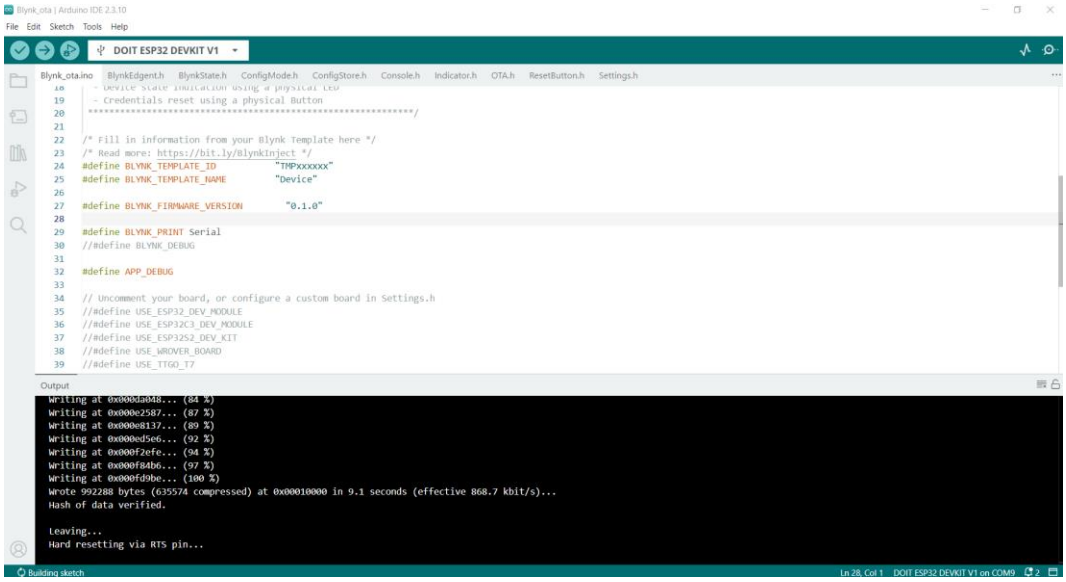
Gambar 2. Daftar datastream V0 sampai V6 pada template



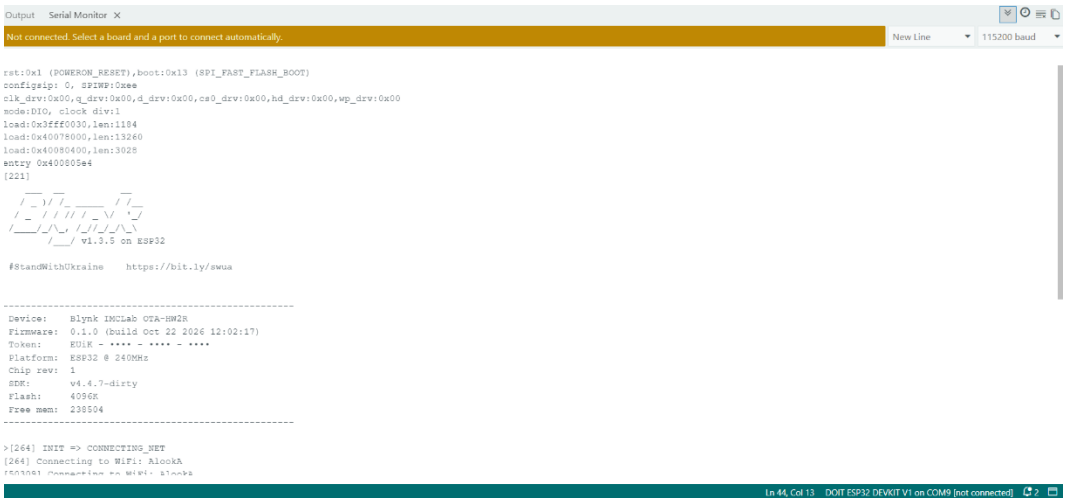
Gambar 3 Tampilan dashboard web pada template "IMCLab OTA"

2. Firmware Awal dan Provisioning (Versi 0.1.0)

   Firmware Blynk_ota berhasil dikompilasi dan diunggah ke ESP32 melalui kabel USB (Gambar 4). Setelah perangkat dimulai ulang, Serial Monitor menampilkan banner perangkat seperti pada Gambar 5. Nilai versi firmware 0.1.0 sesuai dengan definisi BLYNK_FIRMWARE_VERSION pada sketch.



Gambar 4. Proses unggah firmware awal Blynk_ota versi 0.1.0



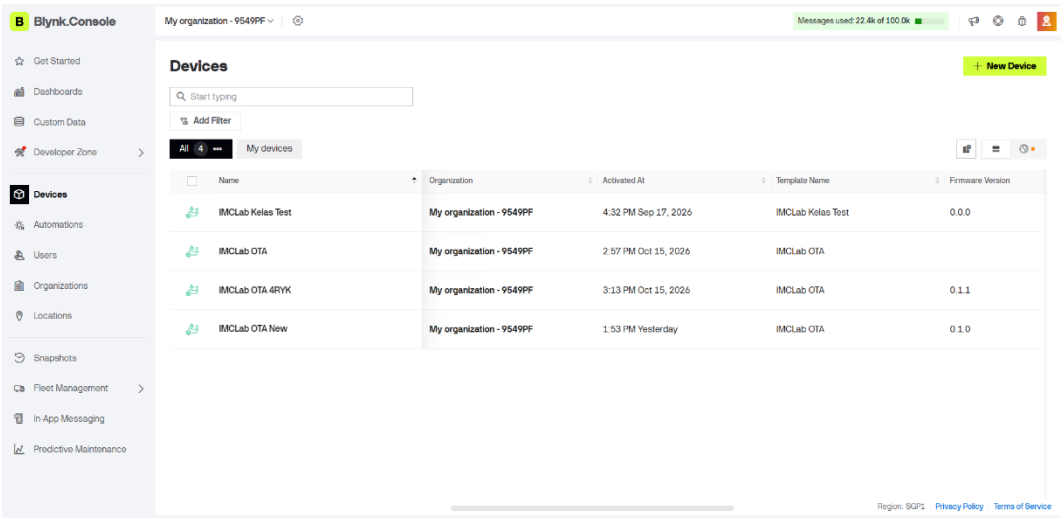
Gambar 5. Banner perangkat pada Serial Monitor setelah firmware 0.1.0 dijalankan

Karena perangkat belum memiliki konfigurasi tersimpan, ESP32 masuk ke mode WAIT_CONFIG dan membuat hotspot Blynk IMCLab OTA-XXXX. Proses provisioning melalui aplikasi Blynk ditunjukkan pada Gambar 6.

Setelah kredensial diterima, perangkat melewati urutan status sebagai berikut. Pada tahap INIT => WAIT_CONFIG, konfigurasi belum ada sehingga ESP32 membuat hotspot dan menunggu aplikasi. Saat WAIT_CONFIG => CONFIGURING, aplikasi meminta /board_info.json sehingga proses konfigurasi dimulai. Selanjutnya pada CONFIGURING => SWITCH_TO_STA, kredensial diterima melalui /config dan hotspot dimatikan. Memasuki SWITCH_TO_STA => CONNECTING_NET, ESP32 menyambung ke router WiFi dan menampilkan pesan “Connecting to WiFi”. Kemudian pada CONNECTING_NET => CONNECTING_CLOUD, WiFi tersambung dan menampilkan pesan “Using Dynamic IP”, lalu perangkat melanjutkan koneksi ke cloud Blynk. Terakhir, pada CONNECTING_CLOUD => RUNNING, perangkat berhasil tersambung ke blynk.cloud dan konfigurasi disimpan ke flash. Setelah melewati seluruh urutan tersebut, perangkat akhirnya mencapai status RUNNING. Pada Blynk.Console, perangkat tampil berstatus Online dengan versi firmware 0.1.0 (Gambar 7).



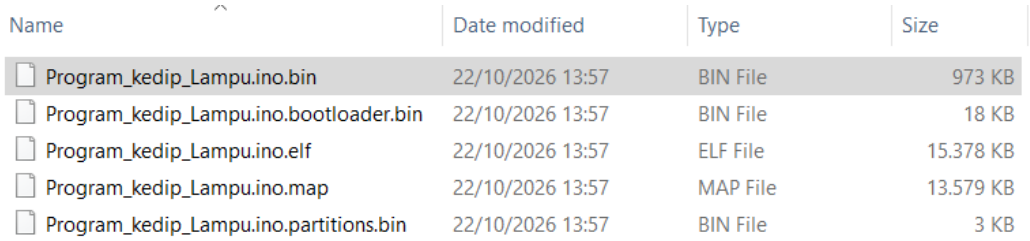
Gambar 6. Proses provisioning perangkat melalui aplikasi Blynk



Gambar 7. Perangkat berstatus Online dengan firmware versi 0.1.0

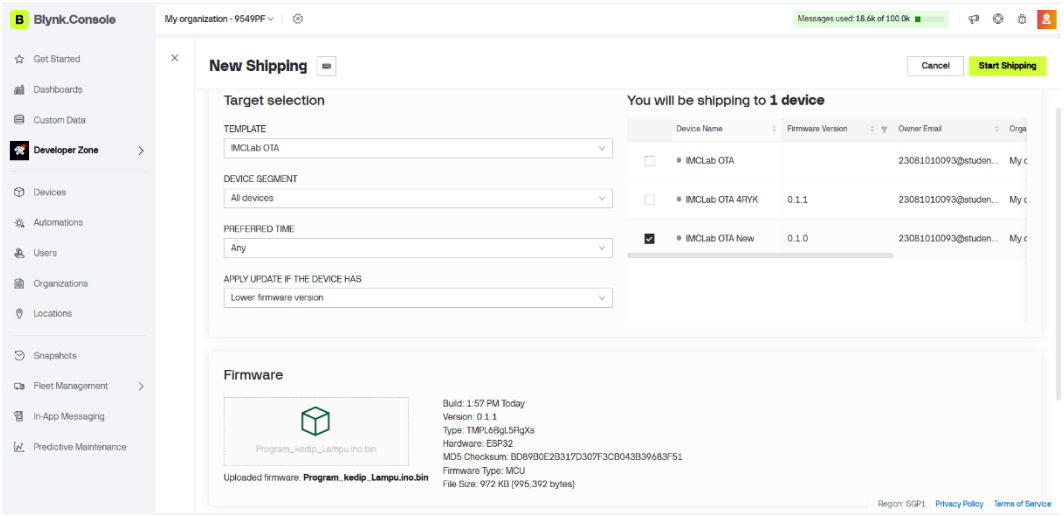
3. Pembaruan Firmware melalui Blynk.Air

Firmware versi 0.1.1 dikompilasi dan diekspor menjadi berkas biner seperti terlihat pada Gambar 8. Ukuran biner sebesar 995.392 byte (sekitar 973 KB) atau sekitar 75,9% dari kapasitas satu slot aplikasi 1.280 KB, sehingga muat pada partisi OTA.

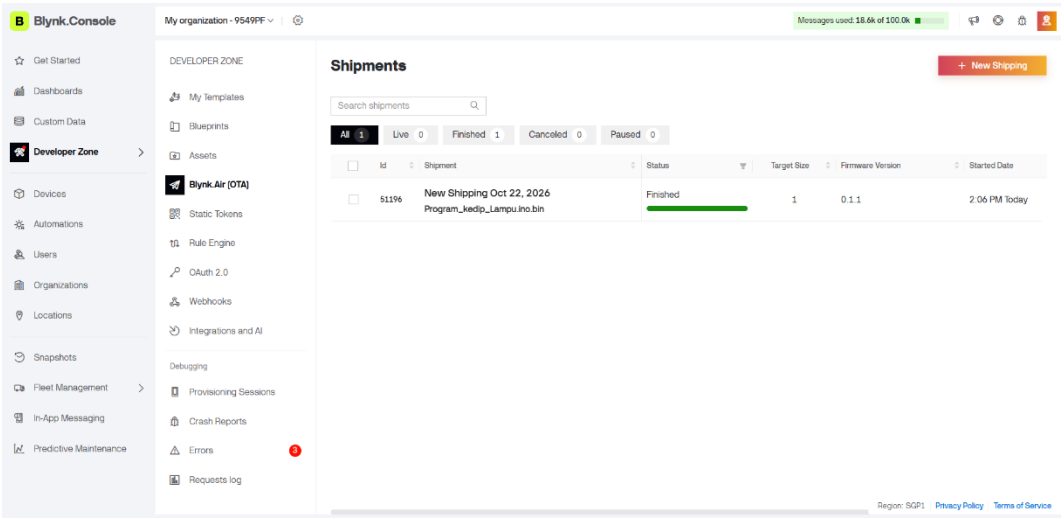


Gambar 8. Berkas biner firmware versi 0.1.1 hasil Export Compiled Binary

Berkas biner tersebut diunggah pada menu Developer Zone > Blynk.Air (OTA), dan Blynk Cloud berhasil membaca informasi firmware dari biner (Gambar 9). Setelah pengiriman dimulai, status pengiriman pada Blynk.Console berubah hingga selesai (Gambar 10).



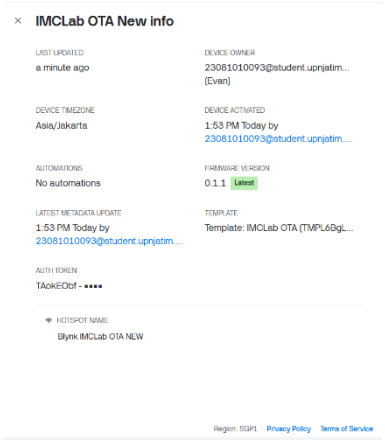
Gambar 9. Pengunggahan firmware versi 0.1.1 pada menu Blynk.Air (OTA)



Gambar 10. Status pengiriman firmware pada Blynk.Console

4. Verifikasi Versi Firmware

Setelah restart, perangkat kembali terhubung ke Blynk tanpa proses provisioning ulang. Serial Monitor menampilkan teks “iMCLab + Blynk Edgent - versi 0.1.1” dan banner perangkat dengan versi 0.1.1, sedangkan Blynk.Console menampilkan versi firmware yang telah berubah (Gambar 11).



Gambar 11. Perangkat menjalankan firmware versi 0.1.1 setelah OTA

5. Pengujian Fungsi Baru

Fungsi Lampu, Mode Kedip, dan Kecepatan Kedip yang baru tersedia pada versi 0.1.1 diuji melalui dashboard (Gambar 12 dan Gambar 13). Kecepatan kedip ditentukan oleh fungsi blinkInterval() yang memetakan nilai slider 1 sampai 10 menjadi jeda 1.000 sampai 100 milidetik. Nilai jeda teoretis dan kolom pengamatan disajikan pada Tabel 3, sedangkan ringkasan pengujian seluruh fungsi disajikan pada Tabel 4.

Tabel 3. Hubungan nilai slider Kecepatan Kedip dan jeda kedip

Nilai slider (V6)	Jeda (ms)	Periode satu siklus (ms)	Frekuensi (Hz)	Pengamatan
1	1.000	2.000	0,50	5
3	800	1.600	0,63	6
5	600	1.200	0,83	7
7	400	800	1,25	12
10	100	200	5,00	54

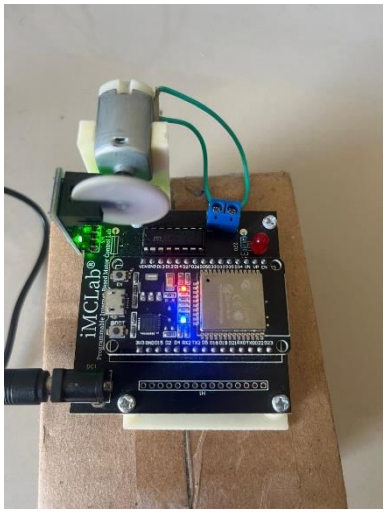
Tabel 4. Hasil pengujian fungsi pada firmware versi 0.1.1

Pengujian	Hasil yang diharapkan	Hasil pengamatan
Switch Lampu (V4) ON	LED biru GPIO 2 menyala terus	Sesuai
Switch Lampu (V4) OFF	LED biru padam	Sesuai
Switch Mode Kedip (V5) ON	LED berkedip sesuai nilai V6	Sesuai

Pengujian	Hasil yang diharapkan	Hasil pengamatan
Mode Kedip dan Lampu ON bersamaan	LED berkedip (Mode Kedip diprioritaskan)	Sesuai
Switch Power (V3) ON	Motor berputar dengan PWM 200 (nilai bawaan)	Sesuai
Slider PWM (V2) = 189	Motor berputar, gauge RPM dan PWM terisi	RPM = 5.880, PWM = 189
Switch Power (V3) OFF	Motor berhenti, RPM kembali 0	Sesuai



Gambar 12. Dashboard Blynk saat fungsi baru diuji



Gambar 13. Kondisi fisik perangkat saat fungsi Lampu, Mode Kedip, dan motor diuji

6. Analisis

1. Kredensial terpisah dari kode. Berbeda dengan metode konvensional yang rawan risiko jika kredensial ditanam langsung di kode sumber, pendekatan menggunakan Blynk.Edgent memisahkan informasi sensitif tersebut. Auth Token dan kata sandi WiFi diberikan saat provisioning dan disimpan di flash memori, sehingga kode sumber hanya berisi Template ID dan Template Name. Hal ini memungkinkan pengguna untuk mengganti jaringan WiFi tanpa harus mengubah dan mengunggah ulang kode program.

2. Konfigurasi bertahan setelah OTA. Tabel partisi pada biner menunjukkan area NVS (tempat konfigurasi disimpan) terpisah dari slot aplikasi app0 dan app1. Proses OTA hanya menulis ke slot aplikasi sehingga SSID, kata sandi, dan token tidak terhapus. Hal ini menjelaskan mengapa perangkat kembali Online setelah restart tanpa provisioning ulang.

3. Keamanan terhadap kegagalan pembaruan. Seluruh kegagalan pada enterOTA() mengarah ke MODE_ERROR, kemudian enterError() menunggu 10 detik dan memulai ulang perangkat. Karena Update.end() belum berhasil, slot yang aktif tidak berubah dan perangkat tetap menjalankan firmware lama. Konsol Edgent juga menyediakan perintah firmware rollback untuk kembali ke slot sebelumnya.

4. Kapasitas penyimpanan. Biner 995.392 byte memakai sekitar 75,9% dari satu slot 1.280 KB. Margin sekitar 24% menunjukkan firmware masih dapat diperbesar, tetapi Update.begin() akan menolak biner yang melebihi ruang tersedia dengan pesan “Not enough space to begin OTA”.

5. Perilaku motor saat OTA. Saat enterOTA() berjalan, loop() tidak dieksekusi sehingga logika aplikasi berhenti, sementara pembangkit PWM pada ESP32 berjalan di perangkat keras. Motor yang sedang berputar dapat tetap berputar pada duty terakhir selama proses. Selain itu, setelah restart BLYNK_CONNECTED() menyinkronkan V2 dan V3 sehingga motor dapat berputar kembali otomatis jika slider masih berada pada nilai tertentu. Karena itu, Switch Power sebaiknya dimatikan sebelum OTA, sebagaimana pada langkah prosedur 4.6.

6. Kesimpulan

1. Konfigurasi Template "IMCLab OTA", datastream (V0–V6), dan dashboard pada Blynk.Console berhasil diselesaikan. Firmware awal versi 0.1.0 berbasis Blynk.Edgent berhasil diunggah ke ESP32 dan berstatus Online setelah melalui proses provisioning via aplikasi Blynk, tanpa perlu menanamkan kredensial di dalam kode sumber.
2. Pembaruan firmware ke versi 0.1.1 (Program_kedip_Lampu) berhasil dilakukan secara nirkabel (Over-The-Air) melalui Blynk.Air tanpa menggunakan kabel USB. Keberhasilan ini dibuktikan dengan adanya log OTA, proses restart otomatis pada perangkat, serta perubahan versi firmware dari 0.1.0 menjadi 0.1.1.
3. Fitur baru seperti kontrol Lampu, Mode Kedip, dan pengaturan Kecepatan Kedip (melalui V4–V6) berhasil dioperasikan dari dashboard. Selain itu, fungsi kontrol motor dan pembacaan nilai RPM yang telah diimplementasikan sebelumnya tetap berjalan dengan baik pada firmware terbaru.
4. Alur kerja Blynk.Edgent dikendalikan oleh mesin status (state machine) yang terdiri dari sembilan kondisi. Proses provisioning dikelola oleh ConfigMode.h dan ConfigStore.h, sedangkan proses OTA ditangani oleh OTA.h melalui

tahapan unduh, verifikasi, penulisan, dan restart. Sistem ini dirancang sedemikian rupa sehingga jika terjadi kegagalan saat proses OTA, firmware yang sedang berjalan tidak akan mengalami kerusakan (corrupt).